Distributed Auction Platform: Initial Design

Designing (We are here :)	*Implementing*	*Completed*

Author: Justin Liu

- Note: Some design parts are being worked on - mock Web UI, request/response parameters and formats), simulators, and deployment

Background

A digital auction is an online marketplace where a seller lists an item for a fixed period and the market sets the price through competitive bidding. The seller sets a start time, an end time and an opening price, and each new bid must beat the current one, so the price only climbs until the clock runs out. The highest valid bid at close wins.

Business & Technical Value

- **Expanded Buyer Access to Categories:** A low-latency search, filtering, and notification architecture connects buyers directly to specialized categories in real time.
- **Guaranteed Fairness:** Strict timestamp-based or sequence-based bid ordering ensures the absolute highest valid bid wins, eliminating disputed outcomes.
- **High Availability & Scalability:** Designed to handle traffic spikes during auction closing windows using asynchronous event-driven architecture.
- **Optimized Monetization:** Flexible fee structures and streamlined seller onboarding reduce time-to-market for new listings.

1. Introduction & Goals

1.1 Overview

Traditional online auctions often limit buyer access to specialized collections and struggle to maintain consistent state when multiple bidders compete in the final seconds of a sale.

This project delivers a multi-user online auction platform that connects buyers to curated items and manages the full lifecycle of timed auctions. Sellers can list items and host auctions, while buyers can browse catalogs, track active lots, and place competitive bids. Also, to ensure fair outcomes, the platform enforces strict bid ordering and automatically extends closing times when late bids arrive to avoid **bid sniping**. The system also includes built-in traffic simulators to generate concurrent bidding activity, allowing us to validate system behavior under load without manually generating users and behaviors.

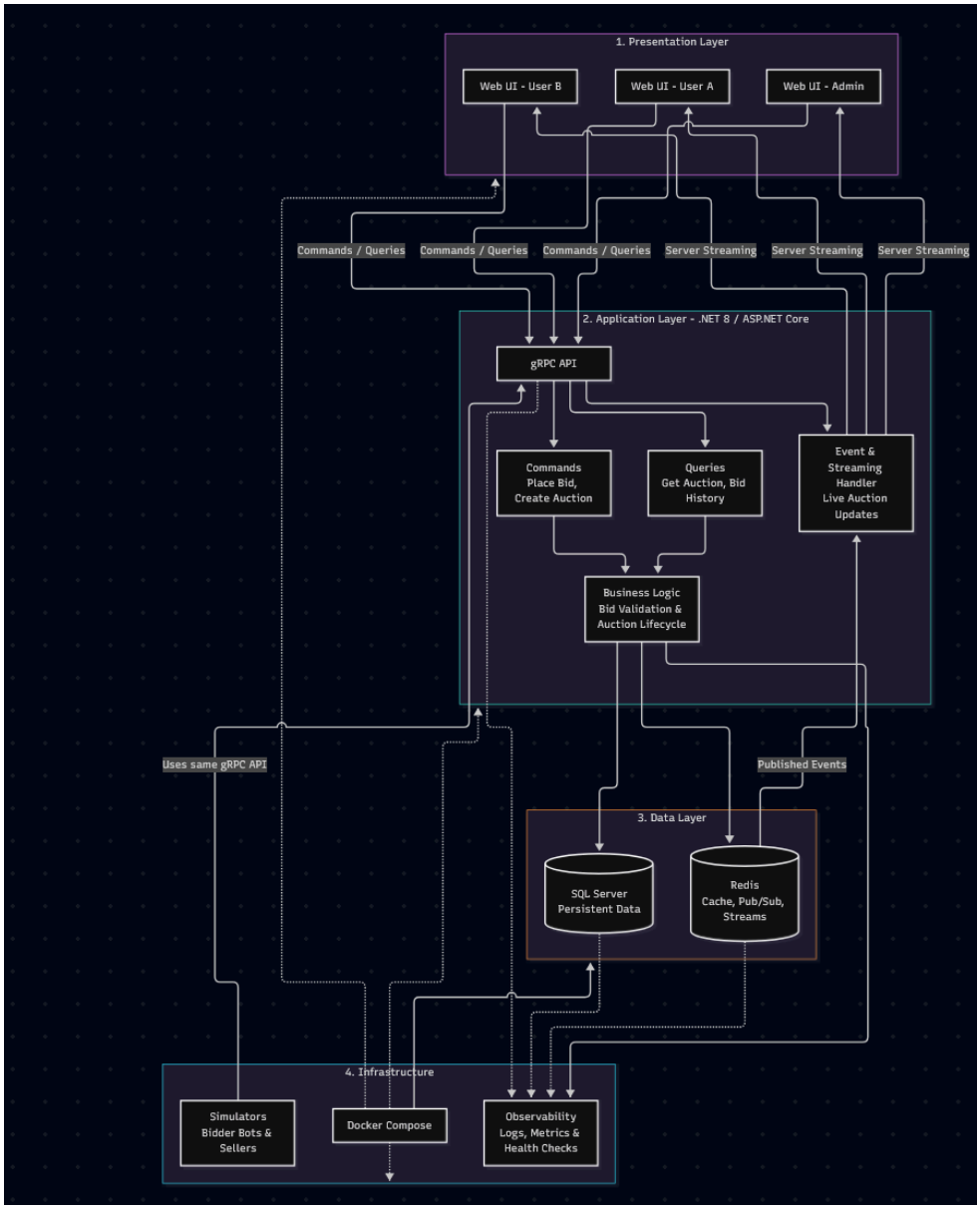
1.2 Success Criteria

- **Near Real-Time State Synchronization:** Multiple concurrent user sessions across separate client instances display identical item prices, bid histories, and auction statuses within **500 ms** of any state change.
- **Bid Integrity Under Load:** The system processes concurrent bids without lost, duplicated, or out-of-order entries.
- **Automated Auction Lifecycle:** Auctions automatically open at their scheduled start time, extend their closing window when a valid bid is placed within the final seconds (e.g., the last 30 seconds), close when time expires, and settle the winning bid.
- **Reproducible Local Deployment & Load Simulation:** A single command launches the entire platform alongside automated bidding simulators to stress-test multi-user concurrency.

1.3 Terminology

- **Lot:** An individual item or bundled set of items listed for sale within an auction.
- **Collection:** A curated group of related lots organized by a seller or theme (e.g., an estate sale, vintage watch series, or specific artist catalog) to simplify buyer discovery.
- **Opening Price (Starting Bid):** The minimum initial amount required to place the first valid bid on a lot.
- **Reserve Price:** An optional, confidential minimum price set by the seller. If bidding closes below the reserve price, the item is not sold, even to the highest bidder.
- **Bid Increment:** The minimum dollar or percentage step by which a new bid must exceed the current highest bid (e.g., if the current bid is $100 and the increment is $5, the next valid bid must be at least $105).
- **Proxy Bidding (Auto-Bidding):** A mechanism where a buyer enters their absolute maximum willingness to pay, and the system automatically places incremental bids on their behalf up to that ceiling whenever they are outbid.
- **Bid Sniping:** The practice of submitting a bid in the final seconds before an auction closes to prevent other buyers from having time to react and place a counter-bid.
- **Anti-Sniping Extension:** A rule that automatically extends an auction's closing time whenever a valid bid is placed inside a defined closing window, ensuring all interested buyers have a fair chance to respond.
- **Out-of-Order / Stale Bid:** An edge case where two buyers submit bids nearly simultaneously, and a bid arrives at the server after the current price has already surpassed it, requiring the system to reject the stale bid cleanly rather than overwriting the higher price.
- **Settlement:** The final lifecycle state after an auction closes where the winning bid is confirmed, the lot is locked against further bidding, and the obligation to purchase is recorded.

Proposed Architecture (High Level)



1.1 Tech Stack

Layer	Technology	Role
UI	Blazor Server + MudBlazor (.NET 8)	Builds the web interface and displays live updates.
Service API	gRPC (ASP.NET Core)	Handles communication between the UI and backend.

Cache and messaging	Redis	Stores frequently accessed data and distributes updates between services.
Database	SQL Server (EF Core + Dapper for hot paths)	Stores permanent application data.
Runtime	Docker + Docker Compose	Runs all application services together.
Observability	Serilog, OpenTelemetry, Health Checks	Tracks errors, monitors performance, and checks service health.

1.1 Core User action Workflow

Workflow 1: Placing a Bid (Critical Path)

1. Client Action (Presentation Layer): User A clicks bid. The MudBlazor UI disables the bid button to prevent double-submitting and sends a gRPC `PlaceBid` request to the backend API containing the `AuctionId`, `UserId`, and `Amount`.
2. API & Validation (Application Layer):
    - The gRPC endpoint receives the payload, validates the request such as checking data types, and passes it to the Business Logic layer.
    - The interceptor validates the JWT signature, checks expiration, and ensures the token has not been tampered with.
    - The business logic checks the Redis Cache for the current highest price and auction status (to avoid hitting SQL Server for high-frequency reads).
        - If not in cache, query from SQL server and repopulate cache
    - Validation Checks: The system verifies that:
        - The auction is currently active (start time $\leq$ current time $<$ end time).
        - The bid amount strictly exceeds the current highest bid plus the required minimum increment.
        - The user is authorized (token has not expired) and has not placed an invalid out-of-order sequence.
3. State Mutation & Persistence (Data Layer):
    - If valid, an atomic operation like a Redis Lua script updates the current price and appends the bid to the bid history.
    - The new bid is durably written to SQL Server, while the active auction state and cache are updated in Redis.
        - If the SQL error fails while Redis succeeds, return a 500 error
4. Anti-Sniping Evaluation:

- The backend checks if the bid arrived within the final closing window, we will be using the last 60 seconds
- If triggered, the auction's official end time is automatically extended by another 60 seconds, and the new end time is updated in Redis and SQL Server.

5. Real-Time Event Broadcast:
    - An event is published to a Redis Pub/Sub / Stream channel.
    - The Event & Streaming Handler picks up the event and pushes the updated price, bid history, and extended clock via gRPC server streaming to all connected UI instances (User A, User B, and Admin dashboards), ensuring sub-second state synchronization.
        - From end-to-end delivery, aim for 300 ms. 999ms may cause race conditions or idempotency issues

Workflow 2: Browsing Categories & Searching Lots

1. Client Action**:** User B navigates to the Categories page in the MudBlazor UI.
2. Query Handling**:** The UI fires a gRPC `GetCategory` query to the Application Layer.
3. Data Retrieval**:**
    - The query handler checks Redis for cached category metadata and active lot listings.
    - For complex, filtered, or paginated catalog queries, the application queries **SQL Server** using indexed fields.
4. Response**:** The server returns the paginated list of lots, current bids, and remaining time back to the client, rendering the catalog instantly.

Workflow 3: Automated Auction Closure & Settlement

When an auction reaches its expiration time without new bids (or after anti-sniping extensions cease):

1. Lifecycle Trigger**:** A background worker or scheduled job evaluates active auctions. When `CurrentTime >= EndTime`, the auction transitions from `Active` to `Pending Settlement`.
2. Winner Determinatio**n:** The system queries the highest valid bid recorded in SQL Server for that lot.
3. State Locking**:** The auction record is locked against further bidding. The winning bid and user ID are flagged as the winning assignment.
4. Notification & UI Update: An event is broadcast via Redis Pub/Sub, updating the UI state for all clients to display "Auction Closed - Winner: User A" in real time.

Workflow 4: Seller Catalog & Lot Creation

When a seller lists a new item for auction, the system validates the metadata, persists it, and indexes it for buyer discovery.

1. Client Action & Authentication**:**
    - The seller fills out the item creation form in the MudBlazor UI (providing title, description, category, starting price, optional reserve price, start time, end time, and metadata).
    - The UI attaches the seller's JWT Bearer token and sends a gRPC `CreateAuctionLot` request to the backend.
2. Authorization & Validation (Application Layer)**:**
    - A gRPC interceptor validates the JWT.
    - The Business Logic layer validates input constraints (e.g., `StartTime < EndTime`, `StartingPrice > 0`, and valid category mappings).
3. Persistence & Caching (Data Layer):
    - The new lot and item details are durably committed to **SQL Server** within a transaction.
    - Basic listing metadata is cached in **Redis** or indexed so buyers can immediately discover the item via catalog search queries.
4. Lifecycle Scheduling & Real-Time Update:
    - The auction scheduler registers the start and end times.
    - An event is broadcast via Redis Pub/Sub to notify connected client catalogs that a new lot has been added to the marketplace.

Workflow 5: Watchlist Subscription & Real-Time Notifications

When a user wants to track a lot and receive alerts for price changes, outbid notices, or closing countdowns:

1. Client Action:
    - The user clicks the "Watch" icon on a lot card in the MudBlazor UI.
    - The client sends a gRPC `ToggleWatchlist` request (`ItemId`, `Action: Add/Remove`) with their JWT Bearer token.
2. Authentication & Persistence:
    - The gRPC interceptor authenticates the user.
    - The backend updates the subscription mapping table in **SQL Server** and registers the user's active streaming connection ID to that item's notification group in **Redis**.
3. Event-Driven Notification Trigger:
    - When a significant event occurs on that lot (e.g., a new bid is placed, an anti-sniping extension is triggered, or the closing window begins), the Event & Streaming Handler checks the Redis subscription index.
    - The system routes a targeted notification payload through the gRPC server stream directly to all active UI sessions subscribed to that lot, instantly updating their watchlist badge, notification drawer, or price ticker.

Workflow 6: Admin Simulator Deployment & Management

To stress-test the system or demonstrate concurrent load without manual intervention, administrators can configure and launch automated bot simulators via the dashboard.

1. Client Action & Admin Authentication:
    - The administrator navigates to the Admin panel in the MudBlazor UI and configures simulation parameters (e.g., number of active bot bidders, bidding frequency, and aggressiveness).
    - The UI sends an authenticated gRPC `ConfigureSimulators` request containing an Admin JWT.
2. Authorization & Command Dispatch:
    - The gRPC interceptor validates the token and strictly checks that the user holds the `Admin` role.
    - The backend dispatches control commands to the **Simulator Service** container (via internal gRPC or a shared message/control channel).
3. Bot Execution & Load Generation:
    - The simulator service spins up automated bot workers.
    - These bots independently authenticate, connect via the public gRPC client API, and execute automated actions (placing concurrent bids, and triggering anti-sniping extensions) to simulate realistic marketplace traffic.
4. Observability & Feedback Loop**:**
    - As the simulators run, the **Observability** stack captures real-time metrics (gRPC request throughput, latency p99, CPU/memory utilization, and Redis cache load).
    - The Admin dashboard streams these metrics back to the UI, allowing operators to monitor system health and concurrency performance live.

1.2 Architecture and Pattern

Client-server architecture

- The web UI and simulators act as clients that communicate with the backend API. The backend processes requests and returns results. This allows multiple users to interact with the same auction system through separate sessions while sharing consistent auction data.

Event-driven communication

- Redis Pub/Sub can broadcast auction events, such as a new highest bid, to interested application instances. This allows connected users to receive updates without repeatedly refreshing the page.

- **Auction Lifecycle Events:** Track transitions of a lot from initial listing through final settlement and broadcast to all users viewing the auction.
  - **`AuctionCreated`:** Fired when a seller publishes a new lot. Instructs connected catalog views to append the new listing without a page reload.
  - **`AuctionOpened`:** Fired by the background scheduler when `CurrentTime >= StartTime`. Transitions the UI from a "Scheduled / Upcoming" countdown to an active bidding state.
  - **`AuctionClosingSoon`:** Fired when the remaining time enters the configured closing threshold (e.g., final 60 seconds). Triggers visual urgency indicators in the UI and alerts watchlist subscribers.
  - **`AuctionExtended`:** Fired when a valid bid arrives inside the closing window (anti-sniping rule). Carries the updated `NewEndTime` so all connected clients synchronize their countdown clocks simultaneously.
  - **`AuctionClosed`:** Fired when `CurrentTime >= EndTime`. Immediately locks the bid input controls across all client UIs while settlement finalizes.
  - **`AuctionSettled`:** Fired once the final winner and closing price are verified in SQL Server. Displays the final outcome (e.g., `Sold` with winning bidder handle, or `Unsold` if a reserve price was not met).
  - **`AuctionCancelled`:** Fired if an administrator or authorized seller voids an active or scheduled auction, clearing active bid states and notifying participants.
- **Bidding & Price Events**
  - **`BidPlaced`:** Fired whenever a valid bid exceeds the current highest bid. Carries the new highest bid, `UserId`, `Timestamp`, and updated `minimum next bid` so all clients update their price tickers and bid history tables instantaneously.
  - **`UserOutbid`:** A targeted user-level event fired alongside `BidPlaced`, routed specifically to the previous highest bidder's active session(s) to trigger an immediate outbid banner or alert.
  - **`BidRejected`:** A targeted event sent only to the submitting client when a bid fails validation during a race condition (e.g., a stale bid arrives milliseconds after another user raised the price). Prompts the UI to display an inline error with the refreshed minimum bid.
- **Simulator & Operational Events: Live in Admin**
  - `SimulatorStatusChanged`: Fired when bot workers are started, paused, throttled, or stopped from the Admin panel, updating active worker counts on administrative screens.
  - `SystemHealthAlert`: Fired when internal health checks detect degraded dependencies (e.g., Redis latency spikes or SQL connection saturation), surfacing real-time warning banners to logged-in administrators.

    - **Catalog & Watchlist Events**
        - These events support personalized tracking and dynamic inventory updates across browsing sessions.
        - `WatchlistItemUpdated`: Fired when a lot on a user's watchlist experiences a price jump, enters its closing window, or closes, updating the user's global notification badge regardless of which page they are currently viewing.

Caching Pattern

- **Data Structures & Caching Patterns**
    - **Active Auction State (Hash):** Active auctions are stored as Redis Hashes (`auction:{id}`) containing current highest price, status, end time, and version counters to support atomic operations.
    - **Bid History Index (Sorted Set - ZSET):** Recent bids for active auctions are maintained in a Redis ZSET (`auction:{id}:bids`), ordered by timestamp or bid amount, allowing $O(\log N)$ range queries and validation checks without querying SQL Server.
    - **Cache-Aside & Write-Through:** Read operations check Redis first; on a cache miss, data is fetched from SQL Server and populated into Redis. Write operations (like placing a bid) update Redis atomically (via Lua scripts) and asynchronously persist to SQL Server
- **Memory Constraints & Sizing**
    - **Memory Footprint:** Assuming an active auction record (metadata + top 50 bids in a ZSET) consumes ~10 KB of RAM, a concurrent window of 5,000 active high-frequency auctions requires approximately **50 MB of raw state data**, comfortably fitting within a standard **2 GB Redis instance**.
    - **`maxmemory` Policy:** The Redis instance is configured with a strict `maxmemory 2gb` ceiling to prevent uncontrolled memory growth on the host container.
- **Eviction Strategy (Hybrid Approach)**
    - **Active Auction & Bidding Keys (`noeviction` / Explicit Lifecycle TTL):** Keys corresponding to active auctions use `noeviction` or are assigned a hard TTL that extends dynamically with anti-sniping rules. They are explicitly deleted or archived to SQL Server *only* after auction settlement.
    - **Catalog & Search Cache Keys (`volatile-lru`):** Non-critical read-through cache keys (such as paginated collection queries or seller profiles) have short TTLs and utilize `volatile-lru` (Least Recently Used with an expiration set) so they can be safely dropped under high memory pressure without affecting live auctions.

Metric	Target (P50 / P99)	Worst Acceptable Limit (P99.9)	Consequence if Breached
Redis Command Latency	$< 1ms \ < 5ms$	$< 15ms$	Warning to Admin
End-to-End gRPC Bid Latency *(Client click to Redis update)*	$< 50ms \ < 150ms$	$< 350ms$	Race condition
Event Broadcast Propagation *(Redis Pub/Sub to UI stream)*	$< 100ms \ < 250ms$	$< 500ms$	Delayed price tickers across clients

Data-access abstraction

- The application can use EF Core and Dapper to separate database access from business logic. EF Core supports general data operations, while Dapper can be used for performance-sensitive queries where needed.

1.3 Observability

- To check that our platform has high availability and rapid incident response, all layers (.NET, gRPC API, Redis, SQL Server, and Docker containers) will have comprehensive health checks, metrics, and automated alerting.
- **2.X.1 Health Checks & Container Down Detection** - Every container managed by Docker Compose includes active health check probes
    - Backend API (/health endpoint): Periodically evaluates database and Redis connectivity. If the backend fails to ping SQL or Redis for 10 consecutive seconds, the container is marked unhealthy.
    - Docker Restart Policies: Containers are configured with restart: unless-stopped (or always in production). If a container crashes due to an unhandled exception or out-of-memory error, the container runtime automatically attempts a restart.
    - Server-Down Alarms: If a container goes down completely, container orchestrators (or log-shipping agents) trigger a critical alert via webhook to an admin (e.g., "Container auction-backend exited with code 137").
- **Key Metrics Tracked** - The application exposes OpenTelemetry-compliant metrics categorized into four core pillars
    - Infrastructure Metrics: CPU usage, memory consumption, and disk I/O per container to spot resource leaks before crashes occur.
    - Application & gRPC Metrics: Request throughput (RPS), P50/P99 latency, and error rates broken down by gRPC method (PlaceBid, GetAuction, etc.).
    - Data Layer Metrics:

        - Redis: Memory fragmentation, hit/miss ratios, active client connections, and Pub/Sub channel backpressure.
        - SQL Server: Active connection pool saturation, deadlocks, and slow-running query durations.
    - Business Logic Metrics: Active concurrent bidders, frequency of anti-sniping auction extensions, and delayed auction settlement jobs.
- **Automated Alarms & Thresholds**

Alert Name	Trigger Condition	Severity	Action
Service unreachable	Container health check fails for > 30 seconds.	Critical	Restart container
High gRPC error rate	HTTP/gRPC 5xx responses exceed 2% of total traffic over a 2-minute window.	High	Alert admin page
Redis / DB disconnect	Backend loses connection to Redis or SQL Server.	Critical	Reconnect logic
Auction settlement lag	An auction passes its end time by > 10 seconds without settling.	Medium	Thread Starvation → Check locking logic and CPU processing

- User Incident Reporting & Error Correlation
    - Error Correlation IDs: Whenever a user encounters an error state (e.g., a rejected bid due to a race condition or network timeout), the UI displays a unique Request ID / Correlation ID.

Domain Model (core)

1. **`User`**
    a. **`userId`** (`UUID` / `Long`): Unique immutable identifier for the user.
    b. **`Username`** (`String`): Unique handle publicly displayed in bid histories and leaderboards (e.g., `Bidder_42`).
    c. **`Email`** (`String`): Unique, verified email address used for login authentication, outbid notifications, and auction win notices.
    d. **`PasswordHash`** (`String`): **Never store plain-text passwords.** This holds a cryptographic hash generated via a modern key-derivation function (can use bcrypt).
    e. **`PhoneNumber`** (`String`, *Optional*): Used for SMS/WhatsApp two-factor authentication or urgent auction closing alerts.
    f. **`Role`** (`Enum`): Defining permissions (`Bidder`,`Seller`, `Admin`).
    g. **`WalletBalance`** (`Decimal`): Preload funds to prevent ghost bidding and scamming.
    h. **`Status`** (`Enum`): Tracks user standing to handle bad behavior:
        - `Active`: Normal standing; fully authorized to browse and bid.
        - `Suspended`: Temporarily restricted due to flagged activity or disputed bids.
        - `Banned`: Permanently blocked from placing bids or creating listings due to malicious behavior (e.g., win-shilling, non-payment, or harassment).
        - `PendingVerification`: Account created via simulator or self-registration awaiting email/identity confirmation
    i. Metadata:
        - **`CreatedAt`** (`Timestamp`): Immutable timestamp recording when the account was registered.
        - **`UpdatedAt`** (`Timestamp`): Tracks the last modification to user credentials, profile, or status.
        - **`ShippingAddress`** (`Object` / `String`): Physical address (Street, City, State, Postal Code, Country) required during post-auction settlement so sellers can ship won lots to buyers.

`2. Category`

- **`CategoryId`** (`UUID` / `Long`, Primary Key): Unique identifier for the category.
- **`Name`** (`String`): Display name of the category (e.g., "Fine Art").
- **`Slug`** (`String`): URL-friendly unique handle for routing (e.g., `fine-art`).
- **`Description`** (`String`, *Optional*): Brief summary of what items belong in this category.
- **`ParentCategoryId`** (`UUID` / `Long`, *Nullable Foreign Key*): References `CategoryId` of the parent node. If `null`, this is a top-level root category.

- **`IsActive`** (`Boolean`): Soft toggle to disable obsolete categories without breaking historical item associations.

3. **`Item`**
    a. **`ItemId`** `(UUID / Long, Primary Key):` Unique identifier for the listing.
    b. **`SellerId`** `(UUID / Long, Foreign Key):` References User.UserId of the seller listing the item.
    c. **`CategoryId`** `(UUID / Long, Foreign Key):` References Category.CategoryId (e.g., Vintage Porcelain Dolls).
    d. **`Title`** `(String):` Concise, searchable headline for the lot.
    e. **`Description`** `(Text):` Detailed provenance, specifications, dimensions, and known flaws.
    f. **`Condition`** `(Enum):` Standardized grading so buyers know what to expect:
        i. `New (Unopened / original packaging)`
        ii. `LikeNew (Mint condition, no visible wear)`
        iii. `UsedGood (Minor cosmetic wear, fully functional)`
        iv. `UsedFair (Visible wear, aging, or minor defects noted in description)`
        v. `ForPartsOrRestoration (Non-functional or damaged collectible)`
    g. **`WarrantyInfo`** `(String / Enum, Optional):` Specifies whether any manufacturer warranty, third-party authenticity guarantee, or AsIs (no warranty) policy applies.
    h. **`EstimatedHandlingDays`** (`Integer` / `String`) (e.g., *"Ships within 2 business days via insured courier"*) and shipping cost structure (`BuyerPays`, `FreeShipping`, or `LocalPickupOnly`).
    i. **`ReturnsAccepted`** (`Boolean`): Explicit flag indicating whether the seller accepts returns (`true` / `false`).
    j. **`ReturnWindowDays`** (`Integer`, *Nullable*): If `ReturnsAccepted` is `true`, defines the return period (e.g., `14` or `30` days) and who covers return shipping
    k. **`Status`** (`Enum`): Tracks the item's state independent of or coupled with the auction (`Draft`, `Published`, `InAuction`, `Sold`, `Unsold`, `Removed`).
    l. **`CreatedAt`** / **`UpdatedAt`** (`Timestamp`): Audit timestamps for listing creation and edits (note: critical fields like `Condition` and `Description` should lock once the first bid is placed to prevent bait-and-switch disputes).
4. **Images**
    a. **`itemId`** `(uuid, Foreign key)`: Links to image(s) of item
    b. **`ImageId`** `(uuid, primary key)`
    c. **`Image`** `(blob)`
5. **`Auction (The Listing & Lifecycle Container)`**

- **AuctionId** (UUID / Long, Primary Key): Unique identifier for the auction event.
- **ItemId**(UUID / Long, Foreign Key): Links directly to the specific item being auctioned.
- **SellerId** (UUID / Long, Foreign Key): Explicitly references User.UserId of the seller (denormalized from the item for rapid authorization checks and permission routing).
- **SequenceNumber** (Long, Monotonically Increasing): A strict order counter per auction ensuring exact event sequencing and gap detection.
- **AuctionType** (Enum): Standard English auction (ascending price), Dutch auction, etc.
- **StartTime** (Timestamp): When bidding opens.
- **EndTime** (Timestamp): Scheduled closing time (dynamically extended if anti-sniping triggers).
- **StartingPrice** (Decimal): The opening floor price.
- **ReservePrice** (Decimal, *Nullable / Confidential*): Minimum price required for the item to actually sell; if final bidding is below this, the item goes unsold.
- **MinimumIncrement** (Decimal): The mandatory minimum step required above the current highest bid.
- fencingToken (Integer) : If worker lock expires and another worker takes over, the stale worker's write is rejected
- **Status** (Enum): Scheduled, Live, Closing, Closed, Settled, Cancelled.
- **CurrentPrice** (Decimal): Denormalized current leading bid amount (cached in Redis and mirrored in SQL for fast read queries).
- **CurrentLeaderUserId** (UUID / Long, *Nullable Foreign Key*): Points to the User.UserId holding the highest valid bid (derived from the top record in the Bid Ledger).
- **Version** (Integer / Long): Optimistic concurrency token used to prevent race conditions during high-frequency bidding wars.
- **CreatedAt** / **UpdatedAt** (Timestamp): Immutable audit timestamps

6. Bid (The Immutable Bid Ledger / Audit Trail)

- **BidId** (UUID / Long, Primary Key): Unique identifier for the individual bid transaction.
- **AuctionId** (UUID / Long, Foreign Key): References Auction.AuctionId.
- **UserId** (UUID / Long, Foreign Key): References User.UserId of the bidder.
- **idempotencyKey** (UUID / String, Unique): Prevent duplicate keys in the case a user submits multiple times
- **Amount** (Decimal): The exact dollar amount bid.

- **Timestamp** (Timestamp): Exact microsecond-precision time the bid was processed and accepted by the server.
- **Status** (Enum): Accepted, Outbid, Rejected_Stale, Rejected_BelowMinimum, Cancelled.

7. Watchlist (User-to-Auction Subscription)

- **watchlistId** (UUID / Long, Primary Key): Explicit unique identifier for the watchlist entry (replacing the ambiguous id).
- **userId** (UUID / Long, Foreign Key): References User.user_id of the buyer following the lot.
- **auctionId** (UUID / Long, Foreign Key): Explicitly references Auction.auction_id so the subscription ties to a specific timed auction event rather than just an unscheduled item.
- **createdAt** (Timestamp): Replaces the generic timestamp field with an explicit creation timestamp recording when the user subscribed (useful for sorting a user's watchlist chronologically).

8. **Settlement (The Master Financial Obligation)**

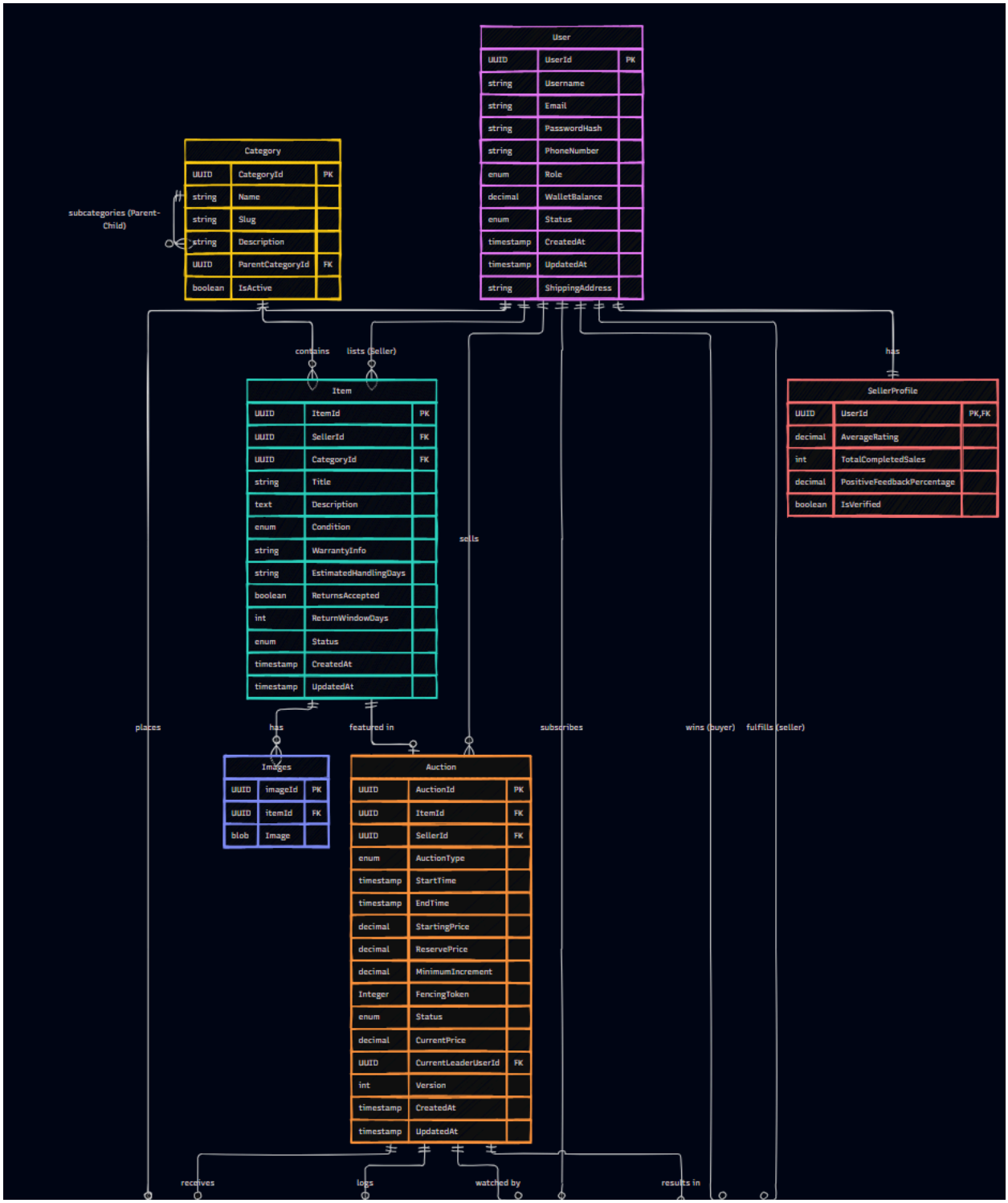
- **settlementId** (UUID / Long, Primary Key): Unique identifier for the post-auction obligation.
- **auctionId** (UUID / Long, Foreign Key): References Auction.auction_id.
- **buyerId** (UUID / Long, Foreign Key): References User.user_id of the winning bidder.
- **sellerId** (UUID / Long, Foreign Key): References User.user_id of the seller.
- **totalAmountDue** (Decimal): Final winning bid plus applicable platform fees/taxes.
- **amountPaid** (Decimal): Running sum of all Approved payments linked to this settlement.
- **installmentsAllowed** (Boolean): Indicates whether split payments are permitted for this lot.
- **status** (Enum): PendingPayment, PartiallyPaid, PaidInFull, Defaulted, Refunded.
- **paymentDadline** (Timestamp): Hard cutoff before the buyer is considered in default.
- **createdAt / updatedAt** (Timestamp): Audit timestamps

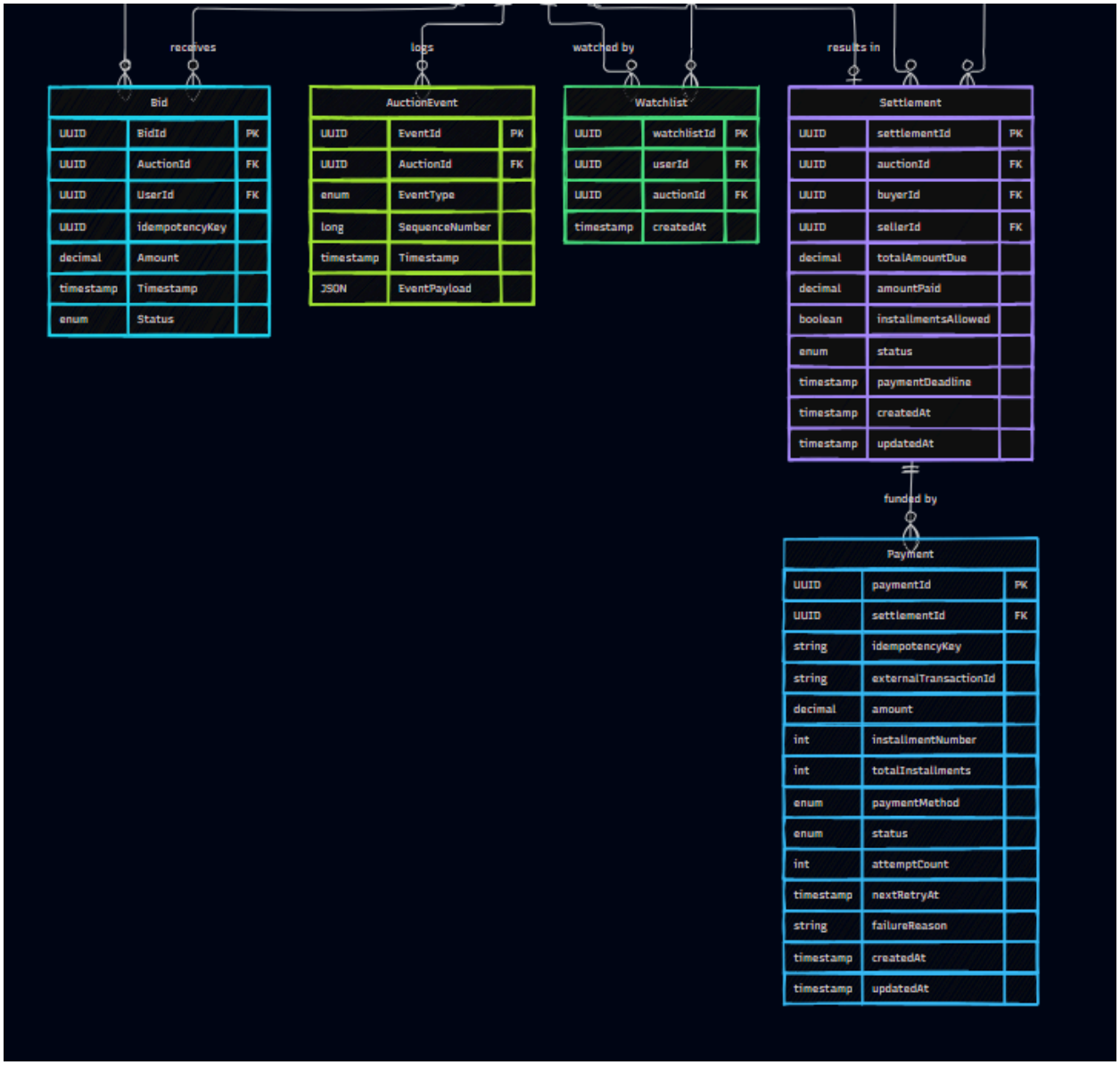
9. Payment (Individual Transaction & Retry Tracker)

- **paymentId** (UUID / Long, Primary Key)
- **settlementId** (UUID / Long, Foreign Key): References Settlement.settlement_id (1-to-Many relationship).
- **idempotencyKey** (UUID / String, Unique): Passed to external payment gateways to prevent duplicate charges during retries.

- **externalTransactionId** (String, *Nullable*): Reference ID returned by the payment provider for reconciliation.
- **amount** (Decimal): Dollar amount of this specific charge or installment.
- **installmentNumber** (Integer): Sequence number of the tranche (defaults to 1 for full single payments).
- **totalInstallments** (Integer): Total expected payments (defaults to 1 for single payments).
- **paymentMethod** (Enum): PlatformWallet, CreditCard, Debit
- **status** (Enum): Pending, Processing, Approved, Declined, Failed_Retrying, Refunded.
- **attemptCount** (Integer): Tracks how many retry attempts have been executed for this payment.
- **nextRetryAt** (Timestamp, *Nullable*): Scheduled time for the background worker to attempt the next retry on a soft decline.
- **failureReason** (String, *Nullable*): Gateway error code or reason (e.g., insufficient_funds, gateway_timeout).
- **createdAt / updatedAt** (Timestamp): Replaces the generic timestamp to track when the attempt was initiated versus when it settled

10. **AuctionEvent**
    a. **eventId** (UUID / Long, primary key): Unique identifier for the specific event record.
    b. **auctionId** (UUID / Long, foreign key): References Auction.auction_id, linking the event to a specific auction lot.
    c. **eventType** (Enum, Not Null): Categorizes the action (AuctionCreated, AuctionOpened, AuctionExtended, AuctionClosingSoon, AuctionClosed, AuctionSettled, AuctionCancelled.).
    d. **sequenceNumber** (Long, Monotonically Increasing): A strict order counter per auction ensuring exact event sequencing and gap detection.
    e. **timestamp** (Timestamp, Microsecond Precision): The exact, unalterable time the event was committed to the ledger.
    f. **eventPayload** (JSON , Not Null): Serialized snapshot of relevant event data (e.g., bid amount, user ID, new expiration time).
11. **Seller Profiles**
    a. **userId** uuid PK, FK
    b. **averageRating** decimal
    c. **totalCompletedSales** int
    d. **positiveFeedbackPercentage** decimal
    e. **isVerified** boolean


User
UUID UserId PK
string Username
string Email
string PasswordHash
string PhoneNumber
enum Role
decimal WalletBalance
enum Status
timestamp CreatedAt
timestamp UpdatedAt
string ShippingAddress
Category
UUID CategoryId PK
string Name
string Slug
string Description
UUID ParentCategoryId FK
boolean IsActive
subcategories (Parent-Child)
contains
lists (Seller)
has
Item
UUID ItemId PK
UUID SellerId FK
UUID CategoryId FK
string Title
text Description
enum Condition
string WarrantyInfo
string EstimatedHandlingDays
boolean ReturnsAccepted
int ReturnWindowDays
enum Status
timestamp CreatedAt
timestamp UpdatedAt
SellerProfile
UUID UserId PK,FK
decimal AverageRating
int TotalCompletedSales
decimal PositiveFeedbackPercentage
boolean IsVerified
sells
places
has
featured in
subscribes
wins (buyer)
fulfills (seller)
Images
UUID imageId PK
UUID itemId FK
blob Image
Auction
UUID AuctionId PK
UUID ItemId FK
UUID SellerId FK
enum AuctionType
timestamp StartTime
timestamp EndTime
decimal StartingPrice
decimal ReservePrice
decimal MinimumIncrement
Integer FencingToken
enum Status
decimal CurrentPrice
UUID CurrentLeaderUserId FK
int Version
timestamp CreatedAt
timestamp UpdatedAt
receives
logs
watched by
results in



1. Currency & Decimal Precision Standard

- **Currency Support**: Stored explicitly tracked via ISO 4217 currency codes (e.g., USD, EUR) at the platform level.
- **Precision**: Financial amounts (StartingPrice, CurrentPrice, Amount, WalletBalance) must use **DECIMAL(12, 4)** or **DECIMAL(15, 4)** in MySQL. This avoids floating-point rounding errors while allowing fractional cent calculations, platform fee breakdowns, supporting numbers up to $999,999,999.9999.

Data Flow

Hot-Path Bid Execution (Redis Lua)

To prevent Time-of-Check to Time-of-Use (TOCTOU) race conditions without paying the latency penalty of distributed locks, all bid validation and state mutation execute inside a single, atomic **Redis Lua script**.

Inputs: `auctionId`, `amount`, `userId`, `idempotencyKey`

Atomic Execution Steps:

1. **Idempotency Check:** Check `SETNX idemp:{auctionId}:{idempotencyKey}`. If present, return the cached response immediately.
2. **Authoritative Time Check:** Fetch Redis server time via `redis.call('TIME')`. Verify `currentTime < auction.endTime`.
3. **State & Business Rule Validation:**
    - Verify `auction.status == 'LIVE'`.
    - Verify `amount >= auction.currentPrice + auction.minIncrement`.
    - Verify `bidderId != auction.currentLeaderUserId`.
4. **State Mutation & WAL Append:**
    - Increment monotonic `sequenceNumber` ($S_n = S_{n-1} + 1$).
    - Update `currentPrice`, `currentLeaderUserId`, and extend `endTime` if within anti-sniping threshold.
    - Atomically append a `BidPlaced` event to a per-partition **Redis Stream** (`XADD`), acting as a durable-in-memory Write-Ahead Log (WAL).

Why a single Lua script instead of distinct APIs?

Splitting validation and mutation across separate network calls allows concurrent requests to read stale prices before a write completes, corrupting auction state. Redis's single-threaded Lua execution guarantees strict serializability per auction shard.

Asynchronous Persistence & Dual-Write Mitigation

Updating Redis and SQL in separate application-level steps creates a split-brain hazard if the API crashes mid-flight. Instead, we treat the Redis Stream as an ingestion buffer for SQL.

1. **Persistence Consumer Group:** Dedicated background workers read from the Redis Stream using `XREADGROUP`.
2. **Idempotent SQL Transaction:** For each event, the worker executes a single SQL transaction:
    - Inserts into `bids` (enforcing `UNIQUE(auction_id, idempotency_key)`).

    - Updates `auctions` using **Optimistic Concurrency Control** (`WHERE id = :id AND sequence_number = :prevSeq`).
3. **Acknowledgement:** Only after the SQL transaction commits does the worker call `XACK` on the Redis Stream.

Why keep SQL Optimistic Concurrency if Redis serializes bids?

Redis is an ephemeral performance layer; SQL is the ultimate source of truth. If a Redis failover loses un-replicated memory or an operator manually intervenes, SQL's `sequence_number` check acts as an invariant guard to reject out-of-order or stale writes.

Real-Time Fan-Out & Client Resynchronization

API instances subscribe to the Redis Stream (or Pub/Sub backed by Stream catch-up) and push updates to connected clients over HTTP/2 gRPC streams.

Sequence Numbers & Gap Recovery
While HTTP/2 guarantees frame ordering within a single TCP connection, sequence numbers are required to survive **client reconnects across different API instances** and dropped packets.

1. **Connection Handshake:** When a client subscribes or reconnects, it sends `SubscribeAuction(auctionId, lastSeenSequence)`.
2. **Gap Detection:** The API instance compares `lastSeenSequence` against the current `sequenceNumber` in Redis:
    - `lastSeenSequence == currentSequence`: Attached directly to live stream.
    - `lastSeenSequence < currentSequence`: Query the Redis Stream (XRANGE) or SQL `bids` table for missing events in range `(lastSeenSequence, currentSequence]`, flush them to the client, and seamlessly transition to live events.
    - `lastSeenSequence` is too old (evicted from stream): Send a full `AuctionSnapshot` payload to reset client state.

Time Authority & Clock Drift Mitigation

Relying on API instance local clocks (`System.currentTimeMillis()`) causes inconsistent bid rejections when instances drift.

- **Server Time Authority:** All expiration checks inside the Redis Lua script use `redis.call('TIME')` (backed by NTP-synchronized hosts). API instances never evaluate auction expiration locally.
- **Continuous Client Calibration:** Every gRPC frame and unary RPC response includes `serverTimestampMs`. The client continuously recalculates its clock offset (

$\Delta t = T_{server} - T_{local}$) using an Exponential Moving Average (EMA) to absorb network jitter and keep UI countdown timers accurate.

Auction Closing & Settlement Worker

A dedicated **Auction Lifecycle Service** monitors expiring auctions via a Redis Sorted Set (`ZSET` scored by `endTimeMs`).

Distributed Locking with Fencing Tokens

To prevent two workers from double-settling an auction if a Garbage Collection pause or network partition causes a simple TTL lock to expire mid-flight:

1. Worker acquires a lease and receives a monotonically increasing **Fencing Token** (T_{fence}) from the coordination store.
2. Any database write during settlement includes `WHERE fencing_token < :newToken`, guaranteeing stale workers are rejected by SQL even if they believe they still hold the lock.

Durability-First State Transition

Flipping Redis to CLOSED before committing to SQL creates a catastrophic failure mode: if SQL fails and Redis later rebuilds from SQL, the auction would reopen.

Strict Closing Order:

1. Worker acquires fenced lock.
2. **Durable Commit (SQL First):** Worker executes SQL transaction updating `auctions.status = 'CLOSED'` and inserting the `settlements` record.
3. **Hot-Path Rejection (Redis Second):** Once SQL commits, the worker executes a Lua script setting `status = 'CLOSED'` in the Redis hash and emits the `AuctionClosed` event.

Note on In-Flight Bids: Any bid that passed the Redis Lua check milliseconds before step 3 will fail the SQL status = 'LIVE' check during async persistence. The persistence worker catches this constraint violation, marks the bid REJECTED_CLOSED, and emits a compensating rejection event to the user.

Failure Modes & Recovery Matrix

Failure Scenario	System Impact	Recovery Mechanism

SQL Write Fails (Transient)	Bids remain in Redis Stream unacknowledged.	Persistence worker retries with exponential backoff; dead-letter queue (DLQ) alerts.
Persistence Worker Crashes	Messages remain in Redis Pending Entries List (PEL).	Sibling workers claim idle messages via XAUTOCLAIM after timeout and process idempotently.
Redis Node Crash / Data Loss	Hot state for active auctions is wiped.	API detects cache miss, acquires a rebuild mutex, queries SQL for the latest `AuctionSnapshot` + `sequenceNumber`, and reforms Redis before accepting bids.
API Instance Crashes	Connected UI clients lose gRPC stream.	Clients auto-reconnect to healthy API instances, passing `lastSeenSequence` to replay missed events without state loss.
Settlement Worker Stalls	Lock TTL expires mid-settlement.	New worker acquires lock with higher fencing token ($T + 1$); stalled worker's late SQL writes are rejected by token check

API Services (Initial - Need to add Requests/Response formats)

1. AuthService (Authentication & Security)

- **Register:** Creates a new user account in the system, validating credentials and persisting user state with default buyer roles and initial standing.
- **Login:** Authenticates user credentials, verifying password hashes and issuing initial JWT access and refresh token pairs.
- **GetProfile:** Retrieves account details, user standing, and profile configurations for the currently authenticated user.
- **RefreshToken:** Essential for issuing new JWTs via secure, HTTP-only refresh tokens without forcing the user to log in again during an active auction.
- **Logout / RevokeToken:** Invalidates the active session token or blacklists the refresh token in Redis.
- **UpdatePassword / InitiatePasswordReset:** Handles secure password updates for active users or triggers a verified password recovery token workflow.

2. Catalog & Category Services

- **ListAuctions:** Retrieves paginated lists of auctions filtered by category.
- **GetAuction:** Fetches complete, detailed metadata for a single auction lot, including current price, item specifications, and lifecycle status.
- **CreateAuction:** Allows verified sellers or administrators to submit a new auction listing into draft or scheduled inventory.
- **SearchAuctions:** Executes high-performance search queries against the catalog index to find items by keywords, attributes, or dynamic filters.
- **UpdateAuction / CancelAuction:** Allows sellers or admins to modify unscheduled or scheduled auctions (rules should lock edits once the first bid is placed).

- **ListCategories / GetCategoryTree:** Returns the hierarchical category structure used for organizing inventory and powering sidebar navigation.
- **ListSellerAuctions:** Fetches all active, upcoming, or past auctions belonging to a specific seller profile.

3. BiddingService (High-Throughput Bidding)

- **BidPlace:** Executes the high-throughput atomic bid placement logic via Redis concurrency gates and the durable outbox pipeline.
- **GetBidHistory:** Returns the chronological list of all valid bids placed on a specific auction lot for auditing and UI rendering.
- **PlaceProxyBid (or Max Bid Support):** In high-end auctions, users prefer setting a maximum ceiling and letting the system automatically outbid competitors up to that limit.
- **GetHighestBid:** A fast, cache-backed read method (reading directly from Redis) for UI polling fallback or initial load rendering.

4. SettlementService (New Service - Post-Auction Lifecycle)

- **GetSettlement:** Fetches the total amount due, fees, tax calculations, and payment status for a won lot.
- **UpdateShippingAddress:** Allows the winning buyer to confirm or update their delivery destination before final payment.
- **SelectInstallmentPlan:** If installments_allowed is true, allows the buyer to configure a split-payment schedule.

5. PaymentService (Financial Transactions)

- **InitiatePayment:** Creates a pending payment transaction record and communicates with external gateway APIs to generate a secure checkout intent.
- **GetPaymentStatus:** Queries the current status of an active or settled payment transaction (e.g., Pending, Approved, Failed).
- **RetryPayment:** Triggers a new attempt to process a failed or declined payment using existing or updated funding sources.

- **RefundPayment:** Submits a request to issue a partial or full refund for a settled financial transaction via the payment gateway.
- **HandleStripeWebhook:** A secure gRPC/HTTP ingress method for asynchronous payment gateway callbacks to update payment status from Processing to Approved or Declined.
    - A third party API will not be used, but a simple class to check for valid / invalid cards can be used as a substitution
- **ListUserInvoices:** Retrieves historical payment receipts and tax invoices for accounting reconciliation.

6. LiveService (Real-Time Streaming)

- **SubscribeAuction:** Opens a real-time gRPC/HTTP/2 stream for a specific auction to fan out price updates, bid events, and closing countdowns.
- **SubscribeUser:** Establishes a private notification channel for user-specific events such as outbid alerts, win notifications, and payment confirmations.
- **Ping:** Bidirectional streaming or built-in keepalive metadata to detect dropped client connections instantly and clean up active Redis subscription channels.

7. WatchService (Watchlist Subscriptions)

- **ToggleWatchlist , ListWatched:** Manages user watchlist bookmarks (add/remove), allowing bidders to track specific auctions or retrieve their current list of saved lots.
- **WatchlistItemUpdated**: Price jump, enters its closing window, or closes, updating the user's global notification badge regardless of which page they are currently viewing.

8. SellerService & AdminService (Management & Moderation)

- **SellerService.PublishItem / UnpublishItem:** Moves items between Draft, Published, and active inventory states.
- **SellerService.GetSellerAnalytics:** Returns total completed sales metrics.
- **SellerService.CreateItem:** Registers a new physical or digital inventory item in the seller's catalog prior to auction scheduling.
- **AdminService.SuspendUser / BanUser / VerifySeller:** Moderation tooling to manage user standing (Status enum in the database schema) and verify seller credentials.
- **AdminService.ForceCancelAuction:** Emergency kill-switch for fraudulent listings or severe system bugs during a live auction.
- **AdminService.SimulatorControl:** Provides control hooks to run traffic tests, speed up timers, or simulate load during staging and evaluation.
- **AdminService.ForceClose:** Manually terminates a live auction ahead of schedule from an administrative control panel.
- **Admin.SystemStats:** Aggregates platform-wide health metrics, active connection counts, and throughput statistics for monitoring dashboards.